

grounding_agent — A 5-minute walkthrough

Vineet Singh

May 13, 2026

CONTENTS

grounding_agent	
The problem	
What we built — at a glance	
Seven dimensions of agent failure	
How we score — two kinds of judges	
The test setup	
Two prompt variants we compare	
Results — the headline	
Results — per-dimension	
What the framework caught	
What doesn't work yet	
What's next	
Production at 100k tasks/day — cost	
Production at 100k tasks/day — cloud infra	
How to find more	

grounding_agent

Evaluating AI agents that use tools — beyond a single pass/fail.

A 5-minute walkthrough.

Vineet Singh — Meraki Labs Founding AI Engineer Work Trial, PS4.

The problem

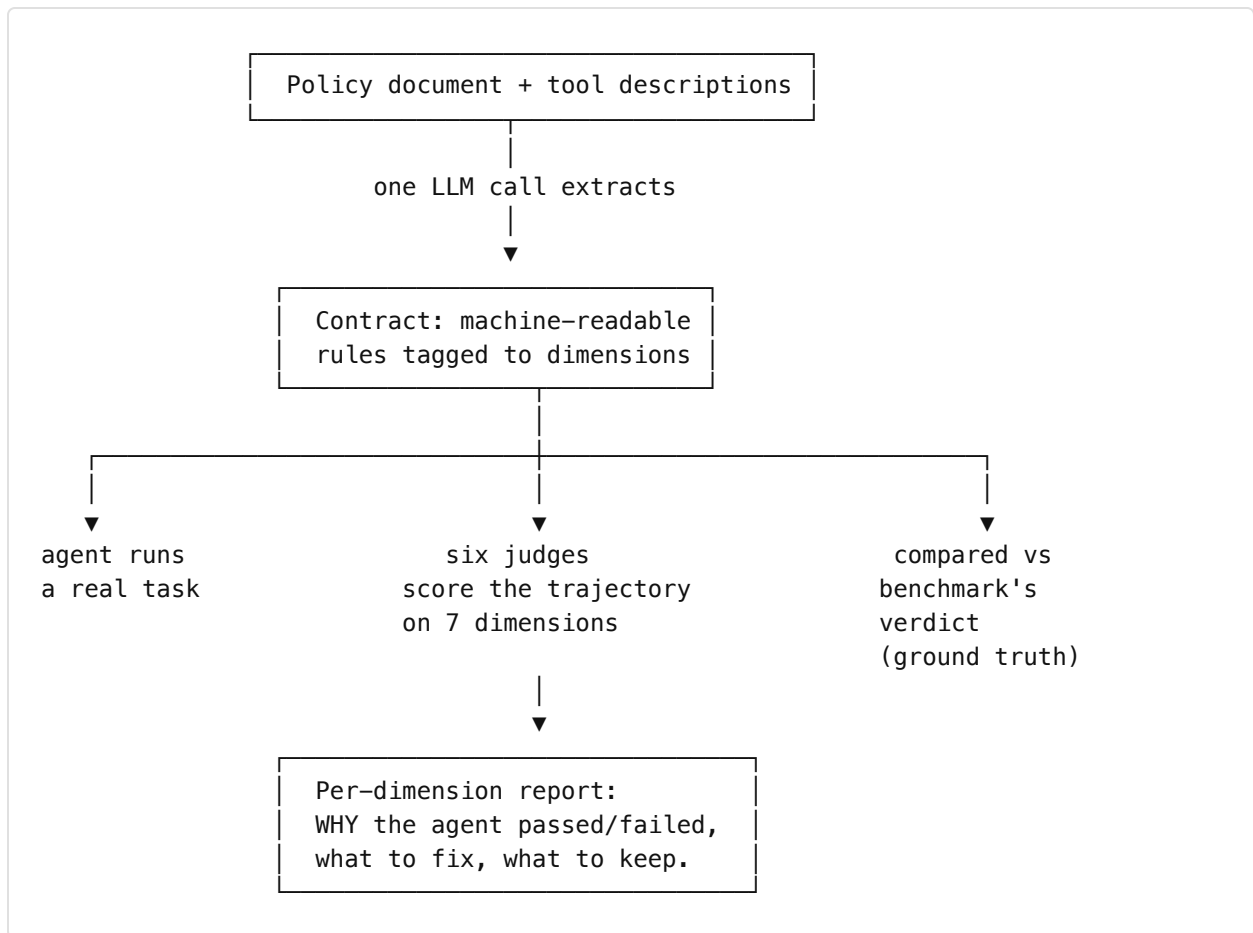
When an AI agent calls tools to do real work (book a flight, cancel a reservation, refund money), a single pass/fail score tells you **that** it failed but not **why**.

Did the agent break a policy rule? Did it call tools in the wrong order? Did it forget to confirm with the user before charging the card? Did it make up information that wasn't in the tools? Did it transfer to a human when it shouldn't have?

A one-number scorecard hides every interesting answer.

This project builds the multi-dimensional answer.

What we built — at a glance



Two-layer evaluation. Mechanical checks for things a computer can verify (was the user’s “yes” present before charging?). LLM judges for things only an LLM can decide (did the agent follow the spirit of the policy?).

Seven dimensions of agent failure

	Dimension	What goes wrong
1	Policy compliance	Agent breaks a business rule (e.g. modifies a non-modifiable flight).
2	Confirmation discipline	Agent acts before asking “is this what you want?”.
3	Information grounding	Agent invents facts not in the tools’ output.
4	Scope adherence	Agent transfers to a human when it should have helped.
5	Tool sequence correctness	Agent calls a write-tool before the matching read-tool.
6	Tool argument correctness	Agent passes the wrong number/id to a tool, causing it to error.
7	Task completion	Agent gets the user’s actual goal done.

Each dimension is its own pass/fail verdict. **Same trajectory, seven independent scores.**

How we score — two kinds of judges

Mechanical judges (3 of 6): small Python checks that walk the trajectory and answer a yes/no question objectively.

- *"Did the user say 'yes' before each mutating tool call?"*
- *"Was every prerequisite read-tool called before its write-tool?"*
- *"Did any tool call come back as an error?"*

These cost almost nothing (<1 ms each) and are 100% reproducible.

LLM judges (3 of 6): an LLM reads the trajectory and the relevant rule, then returns a structured verdict.

- *"Did the agent follow the business policy in spirit?"*
- *"Did the agent only state facts the tools or the user provided?"*
- *"Did the agent correctly decide when to transfer to a human?"*

These cost ~2 seconds each in LLM tokens. They are for things that need judgment, not arithmetic.

The seventh dimension (task completion) is just the benchmark's own pass/fail. We don't re-judge it; we compare our seven dimensions against it.

The test setup

We use the airline customer-support benchmark from Sierra Research (20 tasks total). The benchmark gives us:

- A policy the agent must follow.
- A simulated customer.
- A database the agent can read and modify.
- A pass/fail verdict for each trial.

We split the 20 tasks into two groups of 10:

- **Development tasks (10):** we looked at these while designing our framework — the dimensions, the rules, the prompt variants.
- **Held-out tasks (10):** the framework never saw these during design. Only used to test whether our findings generalize.

This is **not** ML training in the gradient-descent sense. The agent's weights are frozen (gpt-4o-mini). What we “tune” is the **prompt** the agent reads at the start of each conversation.

Two prompt variants we compare

We ran the same agent under two prompt configurations on every task.

v0 — baseline. The policy as the benchmark ships it. No changes.

v2 — with a discipline preamble. v0's policy prefaced by three short rules:

1. Get an explicit user “yes” before any mutation.
2. Read user details before any booking-related write.
3. Only answer with facts from tool outputs or the user.

The question: *Does adding these three reminders help? Does it generalize?*

If v2 improves only on the development tasks, our preamble is over-fit to what we already saw. If v2 also improves on the held-out tasks, the improvement is real.

Results — the headline

	v0 (baseline)	v2 (with preamble)
Overall reward pass-rate	30%	35%
Development tasks	50%	20%
Held-out tasks	10%	50%

v2 beats v0 by 40 percentage points on held-out tasks.

The discipline preamble works — not by memorising specific task patterns, but by getting the agent to follow safer behaviours that transfer to new tasks the framework never saw.

This is exactly the kind of finding multi-dimensional eval should produce: not just “v2 is better” but **where, why, and whether it generalizes**.

Results — per-dimension

dimension	v0	v2
confirmation discipline	70%	70%
information grounding	80%	70%
policy compliance	25%	15%
scope adherence	0%	0%
tool sequence correctness	75%	85%
tool argument correctness	75%	80%

Tool ordering and argument validity improve under v2. The preamble’s “read before write” rule lands directly here.

Policy compliance is hard for both — most failures are subtle business rules the agent slips on.

Scope adherence is stuck at 0%. Honest finding: see “what doesn’t work yet” below.

What the framework caught

Three concrete things a single pass/fail eval would have missed.

1. v2 generalizes. The +40-point lift on held-out tasks is the clearest signal that the discipline preamble produces transferable behaviour, not memorisation.

2. Argument-correctness is the dominant failure mode. Most agent failures are arithmetic — wrong payment splits, gift-card balances, non-existent users. Our framework’s deterministic check flags these directly from the tool server’s error responses.

3. One LLM judge was provably worse than a 10-line Python check. For confirmation, an LLM judge gave 0% pass rate (it always found something to flag). A simple “did the user say yes before each mutation?” Python check gave 70%. We replaced the LLM judge with the Python check.

What doesn't work yet

Scope adherence is stuck at 0% pass rate. The LLM judge sees every transfer as a violation, regardless of whether transfer was the correct action. We tried three different clause wordings; none fixed it.

Why this is structurally hard: deciding “is this user request in-scope?” requires the LLM to hold the entire policy and tool catalog in working memory and compare them to what the user asked. gpt-4o-mini doesn't do this reliably.

The honest takeaway: some dimensions are too judgment-heavy for LLM judges to do well at this model tier. We document this rather than over-engineer around it.

What's next

Short list, ordered by leverage.

1. **A new dimension** for argument *choice* correctness (right flight, right cabin, right split). The current check catches *invalid* arguments — it doesn't catch suboptimal valid ones.
 2. **Read the benchmark's richer reward signal.** τ^3 -bench grades each task on multiple sub-checks (database state, language assertions, communication). The framework currently only reads the binary roll-up.
 3. **Tier judge models by stakes.** Free deterministic checks on 100% of traffic, cheap LLM on a sample, expensive LLM only on flagged cases.
 4. **Move scope adherence to a panel of judges** or accept it as a noisy dimension.
 5. **Tag-review step** after contract generation. One human pass to catch mistagged rules.
 6. **Multi-trial averaging** at higher N for tighter v0/v2 deltas.
-

Production at 100k tasks/day — cost

Per-trajectory cost today (gpt-4o-mini end-to-end):

component	cost
Agent + simulated user	~\$0.008
3 LLM judges	~\$0.004
3 deterministic checks	~\$0.000
Total per trajectory	~\$0.012

At 100k/day = **~\$1.2k/day**, fully judged.

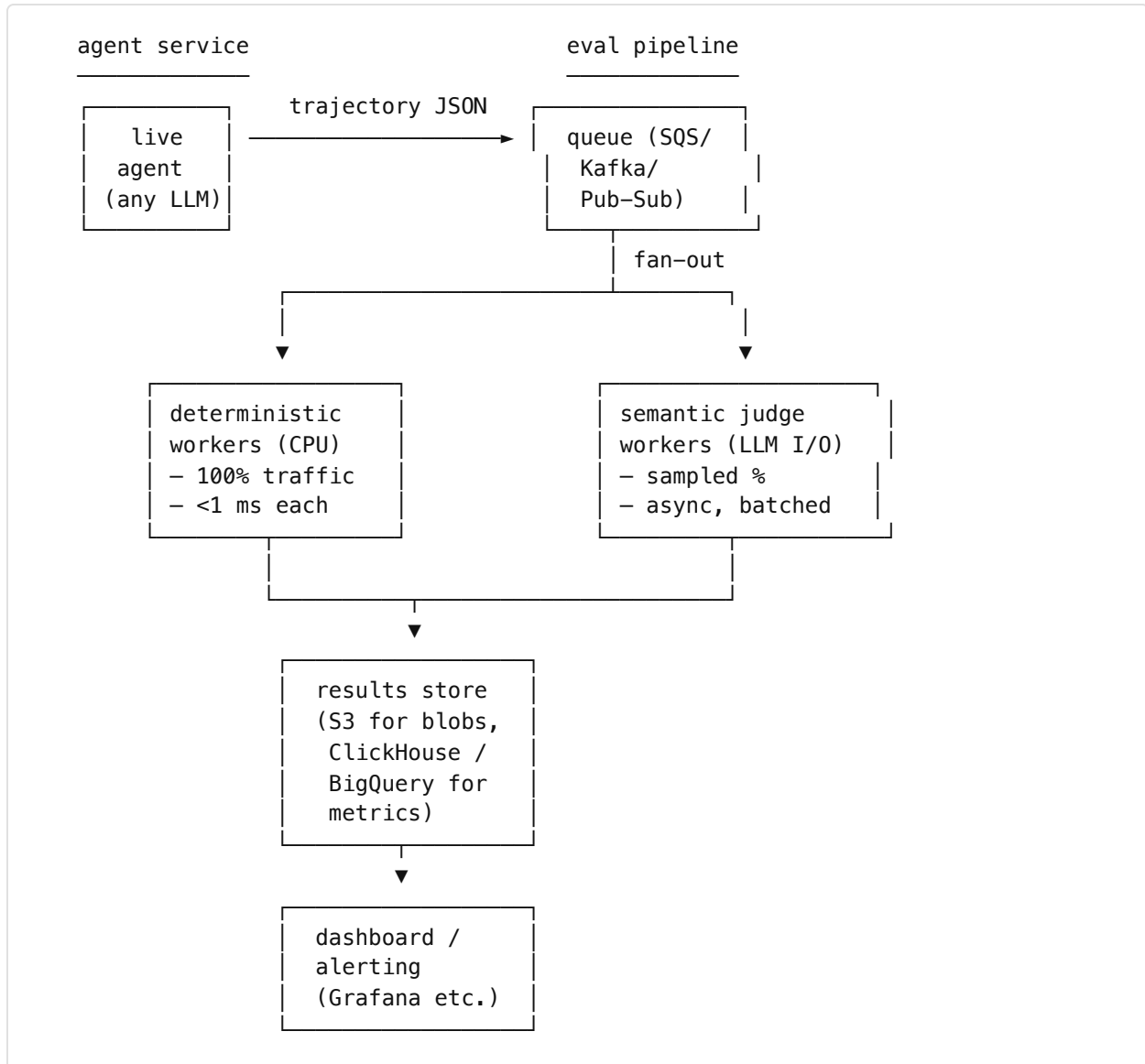
To bring this down without losing signal:

- Deterministic checks on 100% of traffic (essentially free).
- Sample LLM judges to 1–5% of traffic.
- Reserve gpt-4o for arbitration on flagged cases.

Realistic production budget: **~\$50–150/day** for the same signal quality.

Production at 100k tasks/day — cloud infra

A deployment topology that scales:



Sizing knobs:

- **Deterministic workers**: stateless, horizontally scalable, target ~10ms per trajectory. A 10-pod deployment handles 100k/day with 60× headroom.
- **Semantic workers**: bottlenecked on LLM API latency (~2s/call). Run async with a connection pool; ~20–50 concurrent calls suffice at 1% sampling.

- Storage: ~5 KB metrics row + ~30 KB trajectory blob per task. At 100k/day = ~500 MB blob/day + ~500 KB metrics/day. Trivial.

Other infra needs:

- LLM provider rate-limiting (most concerning bottleneck at scale).
 - Region routing for trajectory data residency.
 - Per-tenant isolation (one customer's failures shouldn't poison another's dashboards).
 - Versioned contracts (when policy changes, store the contract id on every result row so old data stays interpretable).
-

How to find more

- `README.md` — clone and run; repo layout; one diagram.
- `WRITEUP.md` — long-form methodology, results, and the reasoning behind each design choice.
- `results/forensics*.md` — three rounds of “what did the data surface, what did we change in response.”
- `tests/` — 125 passing tests; the framework’s invariants enforced.
- `grounding_agent/` — seven Python modules, all under 300 lines.

Repo: `github.com/vinsing09/grounding_agent`.